

### 8.3.3 活动历时估算

活动历时估算直接关系各项具体活动和完成整个项目所需要的总体时间的估算。活动历时估算通常要同时考虑间隔时间。在估算时，要在综合考虑各种资源、人力、物力、财力的情况下，对项目各工作分别进行时间估计。若活动时间估算太短，则在工作中会出现被动紧张的局面；反之，如果活动时间估算太长，则会使整个项目的完工期限延长，从而造成损失。

#### 1. 软件项目的工作量

软件项目的工作量和工期的估算比较复杂，因为软件本身的复杂性、历史经验的缺乏、估算工具的缺乏，以及一些人为错误，导致软件项目的规模估算往往和实际情况相差甚远。因此，估算错误是软件项目失败的主要原因之一。

软件项目通常用代码行（Line of Code, LOC）来衡量项目规模，LOC 指所有可执行的源代码行数，包括可交付的工作控制语言语句、数据定义、数据类型声明、等价声明、输入输出格式声明等。项目经理可以根据对历史项目的审计来核算本企业的单行代码价值。例如，假设某公司每一万行 Java 语言源代码形成的源文件约为 250kB，视频点播系统项目的源文件大小为 3.75MB，则可估计该项目源代码大约为 15 万行，该项目累计投入工作量为 240 人·月，每人·月费用为 10 000 元（包括人均工资、福利、办公费用分摊等），则该项目中 1LOC 的价值为：

$$(240 \times 10\,000) / 150\,000 = 16 \text{ 元 / LOC}$$

该项目的人·月平均代码行数为：

$$150\,000 / 240 = 625 \text{ LOC / 人·月}$$

#### 2. 德尔菲法

德尔菲（Delphi）法是当前比较流行的专家评估技术，该方法结合了专家判断法和三点估算法，在没有历史数据的情况下，对项目进行估算。德尔菲法对减少数据中人为的偏见、防止任何人对结果不适当地产生过大的影响尤其有用，其具体实施步骤如下：

（1）组织者发给每位专家一份软件系统的规格说明书（略去名称和单位）和一张记录估算值的表格，请他们进行估算。

（2）专家详细研究软件规格说明书的内容，对该软件提出三个规模的估算值，即：

$a_i$ ：该软件可能的最小规模（最少源代码行数）。

$m_i$ ：该软件最可能的规模（最可能的源代码行数）。

$b_i$ ：该软件可能的最大规模（最多源代码行数）。

无记名地填写表格，并说明做此估算的理由。在填表的过程中，专家互相不进行讨论，但可以向组织者提问。

（3）组织者为专家们填在表格中的数据进行整理，计算各位专家的估算期望值  $E_i$ ，并综合各位专家估算值的期望中值  $E$ （这种方法通常也称为三点估算法）。

$$E_i = \frac{a_i + 4m_i + b_i}{6} \quad E = \frac{1}{n} \sum_{i=1}^n E_i$$

（4）将各位专家的第一次判断意见汇总，列成图表，进行对比，再分发给各位专家，让专



家比较自己与他人的不同意见，修改自己的估算和判断。然后比较两次估算的结果。若差异很大，则要通过查询找出差异的原因。

(5) 上述过程可重复多次。最终可获得一个得到多数专家共识的软件规模（源代码行数）。在此过程中不得进行小组讨论。在向专家进行反馈的时候，只给出各种意见，但并不说明发表各种意见的专家的具体姓名。

(6) 通过与历史资料进行类比，根据过去完成软件项目的规模和成本等信息，推算出该软件每行源代码所需要的成本。然后再乘以该软件源代码行数的估算值，就可得到该软件的成本估算值。

德尔菲法与常见的召集专家开会、通过集体讨论得出一致预测意见的专家会议法既有联系又有区别。德尔菲法能发挥专家会议法的优点，即能充分发挥各位专家的作用，集思广益，准确性高；能把各位专家意见的分歧点表达出来，取各家之长，避各家之短。同时，德尔菲法又能避免专家会议法的缺点，即权威人士的意见影响他人的意见；有些专家碍于情面，不愿意发表与其他人不同的意见；出于自尊心而不愿意修改自己原来不全面的意见。

德尔菲法的主要缺点是过程比较复杂，花费时间较长。另外，人们无法利用其他参加者的估算值来调整自己的估算值。宽带德尔菲法克服了这个缺点。在专家正式将估算值填入表格之前，由组织者召集小组会议，专家们与组织者一起对估算问题进行讨论，然后专家们再无记名填表。组织者对各位专家在表中填写的估算值进行综合和分类后，再召集会议，请专家们对其估算值有很大变动之处进行讨论，请专家们重新无记名填表。这样适当重复几次，得到比较准确的估计值。由于增加了协商的机会，集思广益，使得估算值更趋于合理。

### 3. 类比估算法

类比估算法适合评估一些与历史项目在应用领域、环境和复杂度等方面相似的项目，通过新项目与历史项目的比较得到规模估计。由于类比估算法估计结果的精确度取决于历史项目数据的完整性和准确度，因此，用好类比估算法的前提条件之一是企业建立起较好的项目后评价与分析机制，对历史项目的数据分析是可信的。

类比估算法的基本步骤如下：

(1) 整理出项目功能列表和实现每个功能的代码行。

(2) 标识出每个功能列表与历史项目的相同点和不同点，特别要注意历史项目做得不够的地方。

(3) 通过步骤(1)和(2)得出各个功能的估计值。

(4) 产生规模估计。

软件项目中用类比估算法，往往还要解决可复用代码的估算问题。估计可复用代码量的最好办法就是由程序员或系统分析师详细地考查已存在的代码，估算出新项目可复用的代码中需重新设计的代码百分比、需重新编码或修改的代码百分比，以及需重新测试的代码百分比。根据这三个百分比，可用下面的计算公式计算等价新代码行：

$$\text{等价 LOC} = [(\text{重新设计 \%} + \text{重新编码 \%} + \text{重新测试 \%})/3] \times \text{已有 LOC}$$

例如，有 10 000LOC，假定 30% 需要重新设计，50% 需要重新编码，70% 需要重新测试，



那么其等价的 LOC 可以计算为：

$$[(30\%+50\%+70\%)/3] \times 10\,000 = 5000\text{LOC}$$

即复用这 10 000LOC 相当于编写 5000LOC 的工作量。

#### 4. 功能点估算法

功能点（Function Point, FP）估算是在需求分析阶段基于系统功能的一种规模估计方法。FP 估算并不是集中于功能上，而是通过研究初始应用需求，确定各种输入、输出、数据文件、查询和外部接口，以及一些复杂度调整值。通常的步骤如下：

- (1) 计算输入、输出、查询、数据文件与接口的数目。
- (2) 将这些数据进行加权乘。
- (3) 根据对复杂度的判断，总数可以用复杂性调节因子进行调整。

统计发现，对一个软件产品的开发而言，功能点对项目早期的规模估计很有帮助。功能点估算法与程序设计语言无关，但涉及的主观因素比较多。例如，各种权值函数的取值等。

#### 5. COCOMO 模型

COCOMO 模型（Constructive Cost Model）是一种结构型估算模型，是一种精确、易于使用的估算方法。在 COCOMO 模型中，考虑开发环境，软件开发项目的总体类型可分为三种：组织型、嵌入型和半独立型。COCOMO 模型按其详细程度分成三级：基本 COCOMO 模型、中间 COCOMO 模型和详细 COCOMO 模型。

(1) 基本 COCOMO 模型。基本 COCOMO 模型是一个静态单变量模型，它用一个已估算出来的源代码行数为自变量的（经验）函数来计算软件开发工作量。具体计算公式如表 8-1 所示。其中， $k\text{LOC}=1000\text{LOC}$ ； $M$  表示开发工作量，单位为人·月； $T$  表示开发进度，单位为月。

表 8-1 基本 COCOMO 模型的工作量和进度公式

| 总体类型 | 工作量                                 | 进度                        |
|------|-------------------------------------|---------------------------|
| 组织型  | $M = 2.4 \times k\text{LOC}^{1.05}$ | $T = 2.5 \times M^{0.38}$ |
| 半独立型 | $M = 3.0 \times k\text{LOC}^{1.12}$ | $T = 2.5 \times M^{0.35}$ |
| 嵌入型  | $M = 3.6 \times k\text{LOC}^{1.20}$ | $T = 2.5 \times M^{0.32}$ |

(2) 中间 COCOMO 模型。中间 COCOMO 模型则在用 LOC 为自变量的函数计算软件开发工作量（此时称为名义工作量）的基础上，再用涉及产品、硬件、人员、项目等方面属性的影响因素来调整工作量的估算。具体计算公式如表 8-2 所示。其中 EAF 为调整因子，取值范围为 0.9 ~ 1.4。

表 8-2 中间 COCOMO 模型的工作量和进度公式

| 总体类型 | 工作量                                                   | 进度                        |
|------|-------------------------------------------------------|---------------------------|
| 组织型  | $M = 3.2 \times k\text{LOC}^{1.05} \times \text{EAF}$ | $T = 2.5 \times M^{0.38}$ |
| 半独立型 | $M = 3.0 \times k\text{LOC}^{1.12} \times \text{EAF}$ | $T = 2.5 \times M^{0.35}$ |
| 嵌入型  | $M = 2.8 \times k\text{LOC}^{1.20} \times \text{EAF}$ | $T = 2.5 \times M^{0.32}$ |